Table of Contents

---

forEach

Overview

- It is higher order function.
- It is used to iterate over array elements and index.
- `forEach` executes a provided function once for each array element
- **Does NOT return anything** (returns `undefined`)
- Cannot be stopped in the middle (no `break` or `return` to exit early)
- Mutates the original array if you modify elements directly

Syntax

```
array.forEach(callback(element, index, array), thisArg)
```

Key Concepts

1. No Return Value

```
const arr = [1, 2, 3, 4, 5];
const output = [];

const result = arr.forEach((element, index) => {
    output.push(element ** 2);
});

console.log(output); // [1, 4, 9, 16, 25]
console.log(result); // undefined - forEach returns nothing!
```

2. Arrow Functions and `this` Context

Problem with Arrow Functions:

```
const obj = {
    multiplier: 10,
    processArray: function(arr) {
        // ❌ This won't work with arrow function
        arr.forEach((element) => {
            console.log(element * this.multiplier); // 'this' is undefined
        });
    }
};
```

Solution with Regular Functions:

```
const obj = {
    multiplier: 10,
    processArray: function(arr) {
        // ☑ This works with regular function
        arr.forEach(function(element) {
            console.log(element * this.multiplier); // 'this' refers to obj
        }, this); // Pass 'this' as thisArg
    }
};
```

When to Use forEach

- When you need to perform side effects (logging, DOM manipulation)
- When you don't need a return value
- When you want to modify the original array elements

---

map

Overview

- **Always returns a new array** with the same length as the original
- It is higher order function.
- It returns new array.
- It will modify the original array.
- Each element is transformed by the callback function
- Cannot be stopped in the middle by default

Syntax

```
const newArray = array.map(callback(element, index, array), thisArg)
```

Key Concepts

1. Always Returns New Array

```
const arr = [1, 2, 3, 4, 5];

const squared = arr.map((element) => {
    return element ** 2;
});

console.log(arr);     // [1, 2, 3, 4, 5] - original unchanged
console.log(squared); // [1, 4, 9, 16, 25] - new array
```

2. Handling Undefined Returns

```
const arr = [1, 2, 3, 4, 5];

const result = arr.map((element) => {
    if (element < 4) {
        return element ** 2;
    }
    // No return statement = returns undefined
});

console.log(result); // [1, 4, 9, undefined, undefined]
```

3. Using with Objects and `this`

```
const obj = {
    power: 3
};

const arr = [1, 2, 3, 4, 5];

const result = arr.map(function(element) {
    return element ** this.power;
}, obj); // Pass obj as thisArg

console.log(result); // [1, 8, 27, 64, 125]
```

forEach vs map Comparison

Feature	forEach	map
Return Value	`undefined`	New array
Original Array	May mutate	Never mutates
Use Case	Side effects	Transformation
Chainable	No	Yes

---

filter

Overview

- Creates a **new array** with elements that pass a test
- The callback function must return a **truthy or falsy value**
- Falsy values are automatically converted to `false`
- It will modify the original array.
- It is higher order function.
- It is used to iterate over array.

Syntax

```
const newArray = array.filter(callback(element, index, array), thisArg)
```

Key Concepts

1. Truthy/Falsy Evaluation

```
const arr = [1, 2, 3, 4, 5];

const filtered = arr.filter((element) => {
    return element < 4; // Returns boolean
});

console.log(filtered); // [1, 2, 3]
```

2. Falsy Values Conversion

```
const arr = [0, 1, 2, "", "hello", null, undefined, false, true];

const truthyValues = arr.filter((element) => {
    return element; // Implicit truthy/falsy check
});

console.log(truthyValues); // [1, 2, "hello", true]
```

3. Complex Filtering

```
const users = [
    { name: "Alice", age: 25, active: true },
    { name: "Bob", age: 17, active: false },
    { name: "Charlie", age: 30, active: true }
];

const activeAdults = users.filter((user) => {
    return user.age >= 18 && user.active;
});

console.log(activeAdults);
// [{ name: "Alice", age: 25, active: true }, { name: "Charlie", age: 30,
active: true }]
```

Method Chaining

Overview

Method chaining allows you to combine multiple array methods in a single expression, where each method returns an array that the next method can operate on.

Key Rules for Chaining

1. Previous method must return an array
2. Methods are executed left to right
3. Each method receives the result of the previous method

Examples

1. Basic Chaining

```
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

const result = arr
    .filter(num => num % 2 === 0)  // [2, 4, 6, 8, 10]
    .map(num => num ** 2)          // [4, 16, 36, 64, 100]
    .filter(num => num > 20);      // [36, 64, 100]

console.log(result); // [36, 64, 100]
```

2. Complex Chaining

```
const products = [
    { name: "Laptop", price: 1000, category: "electronics" },
    { name: "Phone", price: 500, category: "electronics" },
    { name: "Book", price: 20, category: "education" },
    { name: "Tablet", price: 300, category: "electronics" }
];

const expensiveElectronics = products
    .filter(product => product.category === "electronics")
    .filter(product => product.price > 400)
    .map(product => ({
        ...product,
        discountedPrice: product.price * 0.9
    }));

console.log(expensiveElectronics);
```

---

reduce

Overview

- **Most powerful array method** - can return anything (number, string, array, object)
- Reduces an array to a single value through accumulation
- Has an accumulator that carries the result through each iteration
- It is higher order function.
- It is used to iterate and conclude result to a single value.
- It returns single value.

Syntax

```
const result = array.reduce(callback(accumulator, element, index, array),
initialValue)
```

Key Concepts

1. Basic Summation

```
const arr = [1, 2, 3, 4, 5];

const sum = arr.reduce((accumulator, element) => {
    accumulator += element; // Update accumulator
    return accumulator;     // Return updated value
}, 0); // Initial value = 0

console.log(sum); // 15
```

2. Building Arrays

```
const arr = [1, 2, 3, 4, 5];

const filtered = arr.reduce((accumulator, element) => {
    if (element < 4) {
        accumulator.push(element * 10);
    }
    return accumulator;
}, []); // Initial value = empty array

console.log(filtered); // [10, 20, 30]
```

3. Building Objects

```
const arr = ["apple", "banana", "cherry"];

const indexed = arr.reduce((acc, fruit, index) => {
    acc[index] = fruit;
    return acc;
}, {}); // Initial value = empty object

console.log(indexed); // {0: "apple", 1: "banana", 2: "cherry"}
```

4. Finding Maximum/Minimum

```
const numbers = [45, 23, 78, 12, 67, 89, 34];

const max = numbers.reduce((acc, num) => {
    return num > acc ? num : acc;
}, numbers[0]);

console.log(max); // 89
```

reduce Step-by-Step Execution

```
Initial: accumulator = 0, array = [1, 2, 3, 4, 5]

Step 1: accumulator = 0, element = 1 → return 0 + 1 = 1
Step 2: accumulator = 1, element = 2 → return 1 + 2 = 3
Step 3: accumulator = 3, element = 3 → return 3 + 3 = 6
Step 4: accumulator = 6, element = 4 → return 6 + 4 = 10
Step 5: accumulator = 10, element = 5 → return 10 + 5 = 15

Final Result: 15
```

---

reduceRight

Overview

- Works exactly like `reduce` but processes the array **from right to left**
- Useful when the order of processing matters

Syntax

```
const result = array.reduceRight(callback(accumulator, element, index,
array), initialValue)
```

Example

```
const arr = [1, 2, 3, 4, 5];

const rightReduced = arr.reduceRight((acc, element, index) => {
    console.log(`Element at index ${index} is ${element}`);
    return acc + element;
}, 0);

// Output:
// Element at index 4 is 5
// Element at index 3 is 4
// Element at index 2 is 3
// Element at index 1 is 2
// Element at index 0 is 1

console.log(rightReduced); // 15
```

Practical Use Case - String Reversal

```
const chars = ['a', 'b', 'c', 'd'];

const reversed = chars.reduceRight((acc, char) => {
    return acc + char;
}, '');

console.log(reversed); // "dcba"
```

---

flat

Overview

- Flattens nested arrays up to a specified depth
- Creates a new array with sub-array elements concatenated
- Default depth is 1

Syntax

```
const newArray = array.flat(depth)
```

Examples

1. Basic Flattening

```
const arr = [10, [23, 45], 34, [23, 45, 76, 879], 43, 45];

const flattened = arr.flat(1); // depth = 1 (default)
console.log(flattened); // [10, 23, 45, 34, 23, 45, 76, 879, 43, 45]
```

2. Multi-level Flattening

```
const deepArr = [1, [2, 3], [4, [5, 6]], [7, [8, [9, 10]]]];

console.log(deepArr.flat(1));        // [1, 2, 3, 4, [5, 6], 7, [8, [9, 10]]]
console.log(deepArr.flat(2));        // [1, 2, 3, 4, 5, 6, 7, 8, [9, 10]]
console.log(deepArr.flat(Infinity)); // [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

3. Removing Empty Slots

```
const arr = [1, 2, , 4, 5];
console.log(arr.flat()); // [1, 2, 4, 5] - removes empty slots
```

Performance Comparison

Time Complexity

Method	Time Complexity	Space Complexity
forEach	O(n)	O(1)
map	O(n)	O(n)
filter	O(n)	O(k) where k ≤ n
reduce	O(n)	O(1)
flat	O(n)	O(n)

Best Practices

1. Choose the Right Method

```
// ✅ Good: Use forEach for side effects
arr.forEach(item => console.log(item));

// ❌ Bad: Using map without using the result
arr.map(item => console.log(item)); // Creates unnecessary array

// ✅ Good: Use map for transformation
const doubled = arr.map(x => x * 2);

// ✅ Good: Use filter for selection
const evens = arr.filter(x => x % 2 === 0);
```

2. Efficient Chaining

```
// ✅ Good: Filter first, then map (processes fewer elements)
const result = largeArray
    .filter(item => item.active)
    .map(item => item.name);

// ❌ Less efficient: Map first, then filter
const result2 = largeArray
    .map(item => item.name)
    .filter((name, index) => largeArray[index].active);
```

Summary Table

Method	Returns	Mutates Original	Use Case
forEach	`undefined`	No*	Side effects, iteration
map	New array	No	Transformation
filter	New array	No	Selection/filtering
reduce	Any value	No	Accumulation, aggregation
reduceRight	Any value	No	Right-to-left processing
flat	New array	No	Flattening nested arrays

*forEach doesn't mutate the array structure, but you can mutate individual elements

---

Common Patterns and Recipes

1. Count Occurrences

```
const fruits = ['apple', 'banana', 'apple', 'cherry', 'banana', 'apple'];

const count = fruits.reduce((acc, fruit) => {
    acc[fruit] = (acc[fruit] || 0) + 1;
    return acc;
}, {});

console.log(count); // {apple: 3, banana: 2, cherry: 1}
```

2. Group by Property

```
const people = [
    {name: 'Alice', age: 25},
    {name: 'Bob', age: 25},
    {name: 'Charlie', age: 30}
];

const grouped = people.reduce((acc, person) => {
    const age = person.age;
    if (!acc[age]) acc[age] = [];
    acc[age].push(person);
    return acc;
}, {});

console.log(grouped);
// {25: [{name: 'Alice', age: 25}, {name: 'Bob', age: 25}], 30: [{name:
'Charlie', age: 30}]}
```

3. Remove Duplicates

```
const arr = [1, 2, 2, 3, 4, 4, 5];

const unique = arr.filter((item, index) => arr.indexOf(item) === index);
console.log(unique); // [1, 2, 3, 4, 5]

// Or using reduce
const uniqueReduce = arr.reduce((acc, item) => {
    if (!acc.includes(item)) acc.push(item);
    return acc;
}, []);
```

Happy Ending..!!

Avinash Ranjan